

第2章（DFA）2026

知识图谱（RL与词法分析）

Chomsky体系:(3型:RL 2型:CFL 1型:CSL 0型:REL)

中心概念:(论域 符号 字母表 符号串 语言 问题)

RL:(识别器:(FA:(DFA NFA:(ε-NFA)) RE LG:(LLG RLG)) 判定性质:($\tilde{\upsilon}$:($\tilde{\upsilon}_D$ $\tilde{\upsilon}_N$) 模式匹配 生成) 等价性质:(NFA≡DFA RE≡NFA DFA最小化 RG≡RE) RL簇:(封闭性 泵引理 空性 成员性))

词法分析: (σ-DFA scanner() LEX)

RL（Regular Language）正规/正则语言 RG（Regular Grammar）正规/正则文法
CSL（Context-Sensitive Language）上下文敏感/有关语言
REL（Recursively Enumerable Language）递归可枚举语言
FA（Finite Automaton）有穷自动机 $\tilde{\upsilon}$（FA的扩展转移函数）
DFA（Deterministic FA）确定有限自动机 NFA（Non-deterministic FA）非确定...
RE（Regular Expression）正规式/正则表达式 RG（Regular Grammar）正规/正则文法
LLG（Left-Linear Grammar）左线性文法 RLG右线性文法
σ-DFA多语言联合识别器 scanner()扫描器 LEX（词法分析器生成器）

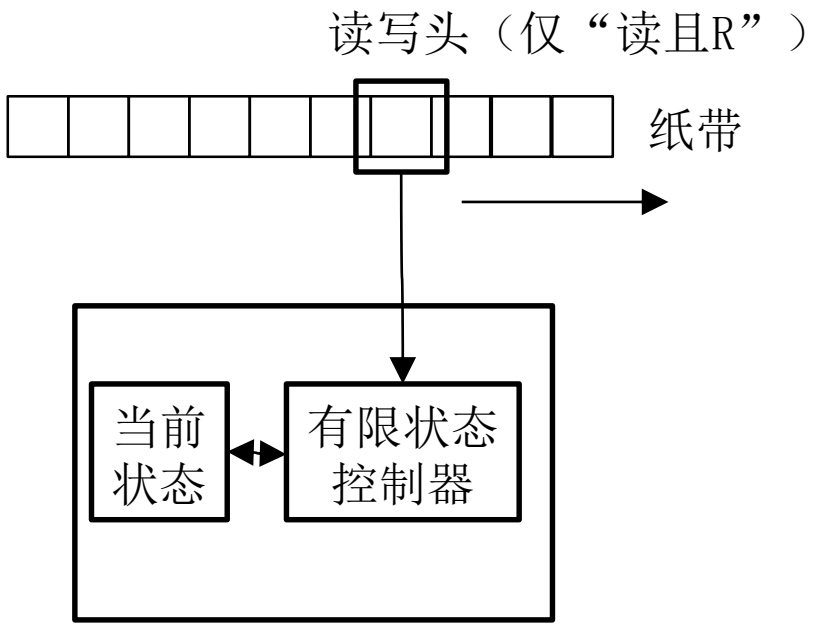
章节安排：RL与词法分析

- 有穷自动机（FA）//第2章
  - 确定有穷自动机（DFA）：完全形、最简形、最小形
  - 非确定有穷自动机（NFA）：NFA、ε-NFA
- 正规式（RE）//第3章
- 正规文法 //第3、4章
  - 左线性文法（LLG）
  - 右线性文法（RLG）

- 多语言联合识别器（σ-DFA）//第4章词法分析的原理
- 词法分析器框架 //第4章词法分析工程落地

2.1 DFA的定义

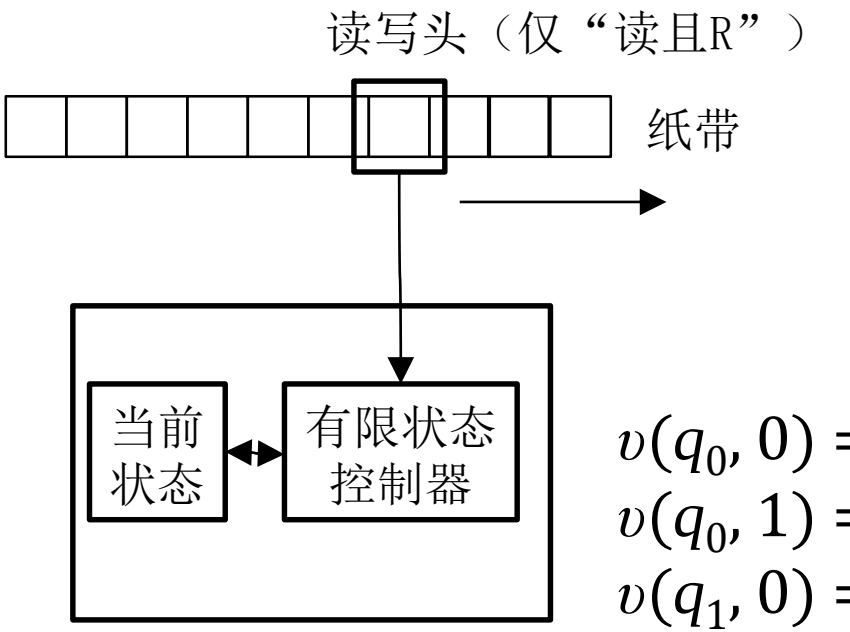
- DFA $A = (Q, \Sigma, \upsilon, q_0, F)$
- 有限状态集合Q
- 字母表Σ为有限符号集合
- 状态转移函数υ: $Q \times \Sigma \mapsto Q$
- $q_0 \in Q$ 初始状态
- $F \subseteq Q$ 接受状态



输入是 Σ 上符号串。DFA对其从左往右逐符号读入。
转移函数 $\upsilon()$ 对 $Q \times \Sigma$ 中每个元素都有返回值。
对于当前状态和当前读入符号，应用 $\upsilon()$ 就得到转移状态。
初始状态恰有 1 个。
接受状态可以没有（若 $F = \emptyset$ 则该DFA接受的语言为$\emptyset$）。
初始状态也可以是接受状态（若, $q_0 \in F$ 则该DFA接受串 ε ）。

2.1 DFA的定义

- DFA $A = (Q, \Sigma, \upsilon, q_0, F)$
- 有限状态集合Q
- 字母表Σ为有限符号集合
- 状态转移函数υ: $Q \times \Sigma \mapsto Q$
- $q_0 \in Q$ 初始状态
- $F \subseteq Q$ 接受状态



$$\upsilon(q_0, 0) = q_0$$
$$\upsilon(q_0, 1) = q_1$$
$$\upsilon(q_1, 0) = q_2$$
$$\upsilon(q_1, 1) = q_1$$
$$\upsilon(q_2, 0) = q_2$$
$$\upsilon(q_2, 1) = q_2$$

例：设计DFA识别“0到多个0后跟1到多个1”。

输入串　0 … 0 1 … 1

q_0 初始状态　q_1 接受状态　q_2

DFA $A = (\{q_0, q_1, q_2\}, \{0, 1\}, \upsilon, q_0, \{q_1\})$

语言 $L(A) = \{0^m 1^n \mid m \geqslant 0, n \geqslant 1\}$

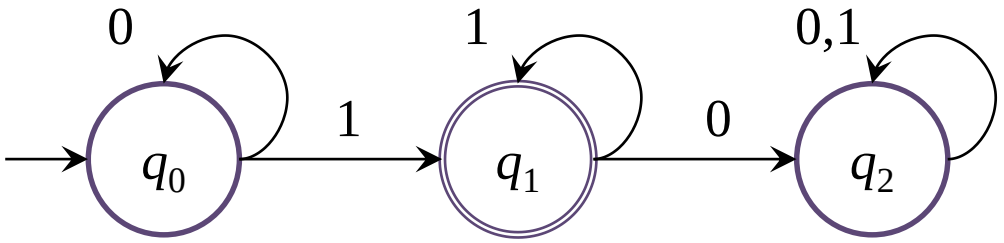
DFA表示：代数、转移图、转移表

DFA $A = (\{q_0, q_1, q_2\}, \{0, 1\}, \upsilon, q_0, \{q_1\})$

$\upsilon = ((q_0, 0), q_0), ((q_0, 1), q_1),$
$((q_1, 0), q_2), ((q_1, 1), q_1), ((q_2, 0), q_2),$
$((q_2, 0), q_2))$

代数

DFA $A = (Q, \Sigma, \upsilon, q_0, F)$
$Q = \{q_0, q_1, q_2\},$
$\Sigma = \{0, 1\},$
$F = \{q_1\}, \quad \upsilon = \ldots$



转移图

	0	1
$\rightarrow q_0$	q_0	q_1
$^*q_1$	q_2	q_1
q_2	q_2	q_2

转移表

$G(A) = (Q, E, \Sigma)$
$E = \{(p, q, a) \mid p, q \in Q, a \in \Sigma, q = \upsilon(p, a)\}$

$\mathbb{T}[\text{ind}(q), 0]$
$\mathbb{T}[0, \text{ind}(a)]$
$\mathbb{T}[\text{ind}(q), \text{ind}(a)]$

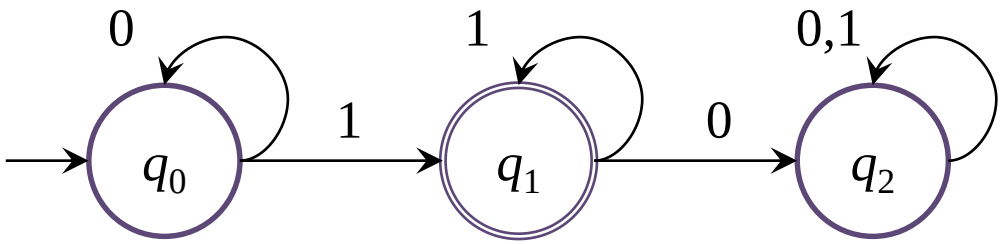
DFA的三种表示都是彼此等价的

代数 A	转移图$\mathcal{G}(A)$	转移表$\mathbb{T}$
Q 元素 q	顶点（圆圈）	$\mathbb{T}[\text{ind}(q), 0]$
Σ 元素 a	权（弧标记）	$\mathbb{T}[0, \text{ind}(a)]$
υ 元素	边（转移弧）	$\mathbb{T}[i, j\,], i, j \neq 0$
q_0	箭头标记顶点	箭头标记$\mathbb{T}[1, 0]$
F 元素	双圈顶点	*标记$\mathbb{T}[i, 0], i \neq 0$

DFA的判定性质: DFA接受、拒绝输入串

- 0、1、00、01、000、001、010、0101

	0	1
$\rightarrow q_0$	q_0	q_1
$^*q_1$	q_2	q_1
q_2	q_2	q_2



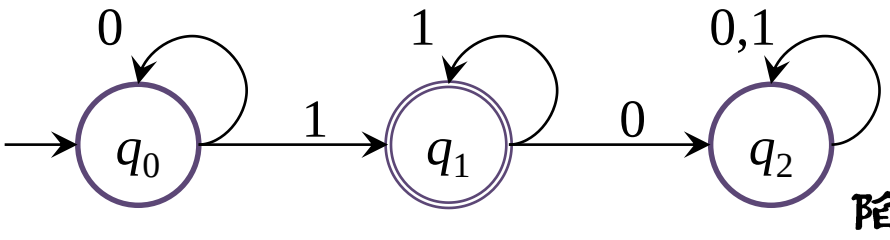
- 输入串，已消耗串，剩余串。
- 当前输入符号，当前状态。
- 初始化输入串为剩余串，剩余串的1-前缀为当前输入符号或 ε 。
- 应用转移函数根据当前状态和当前输入符号得出转移状态，
- 如此运行，直到剩余串为空为止，此时由转移状态属于 F 接受，否则拒绝。

形式描述DFA接受串w（DFA的判定性质）

- DFA $(Q, \Sigma, \upsilon, q_0, F)$ 接受 w，如果对于$r_0, r_1, \ldots, r_n \in Q$ 满足:
  - $r_0 = q_0$
  - $\upsilon(r_i, w_{i+1}) = r_{i+1}$ 其中 $i = 0, \ldots, n-1$
  - $r_n \in F$
- 路径PATH$[r_0, r_n, w]$：$\langle r_0, w_1, r_1, w_2, \ldots, r_{n-1}, w_n, r_n \rangle$

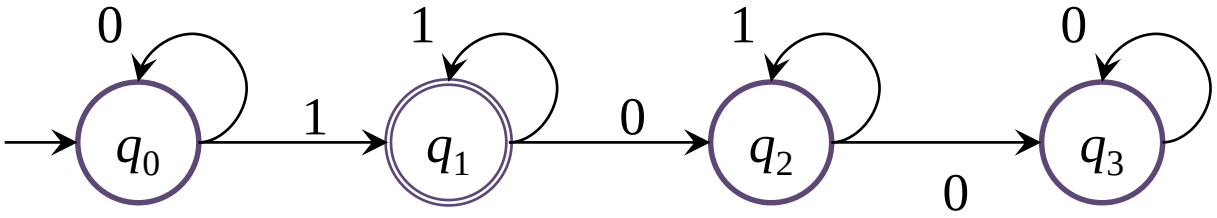
- 对于输入串w，如果DFA中有一条始端为初始状态、末端为接受状态、标记为w的路径，那么该DFA接受w 。
- 这种路径被称为一条有效（成功）路径。
- 对于输入串w，如果不存在有效路径，那么被拒绝。

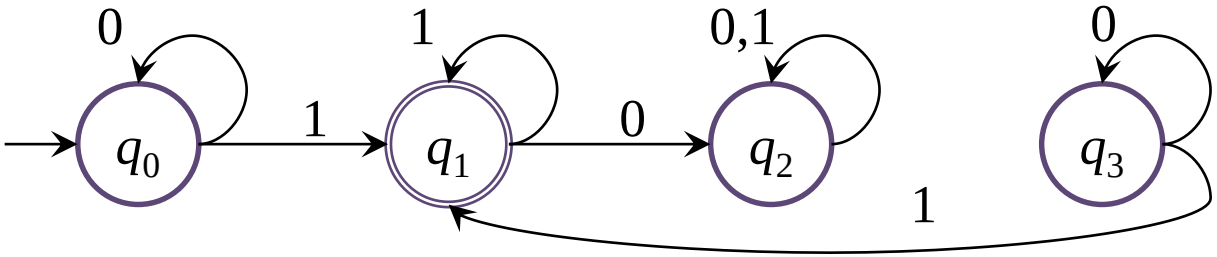
DFA的几种形式（聚焦有效路径）



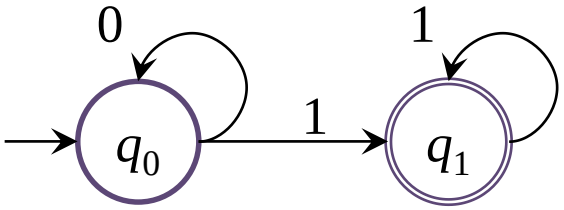
陷阱状态

完全形





完全形

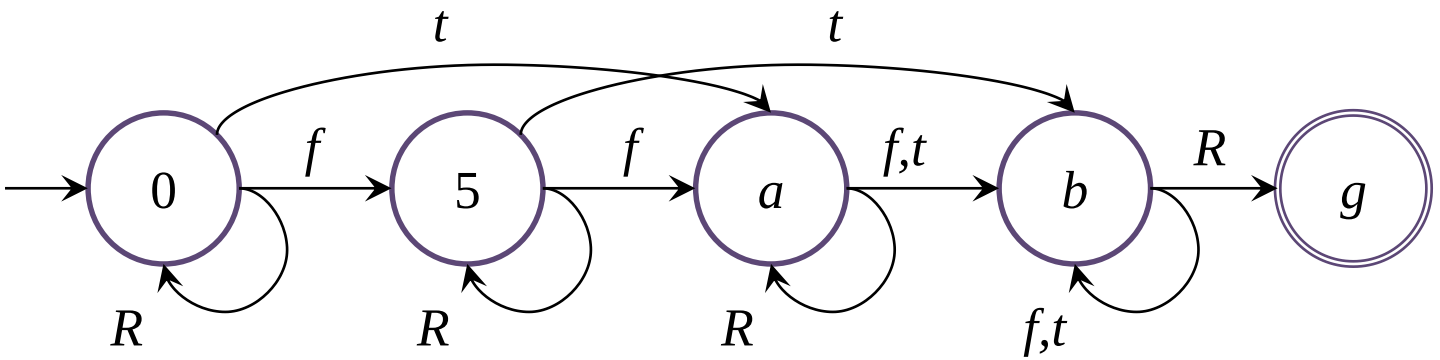


最简形

设计口香糖球售货机的DFA模型

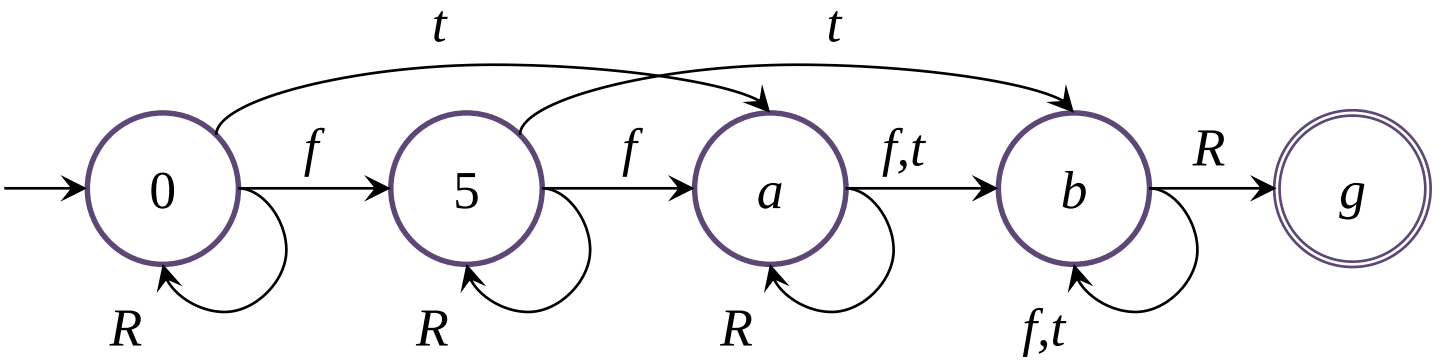
- 状态：记忆投入的钱数并默认机子有糖球可买
- 交互：投入5分、投入10分、按释放键
- 初始状态：投入钱数为0，机子有糖球
- 接受状态：
  - 吐出糖球并找零，或吐不出糖球



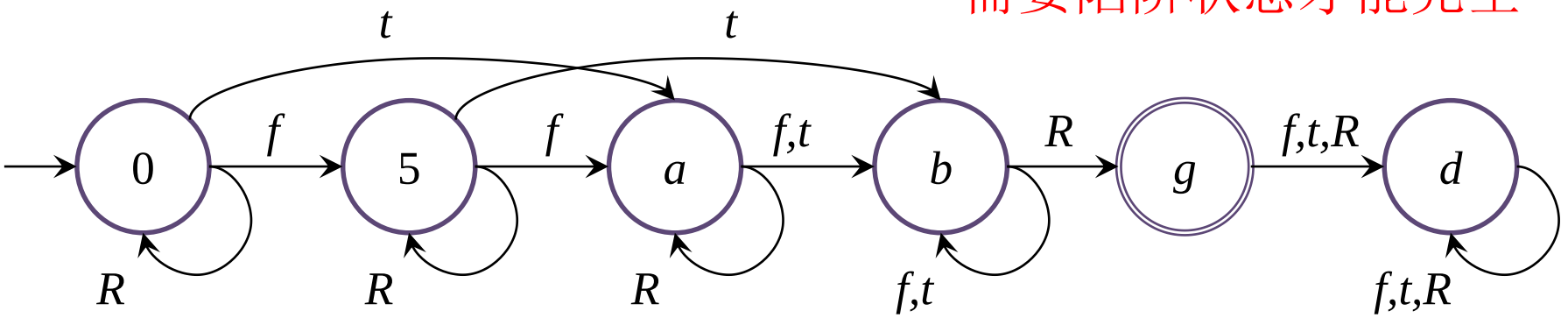


对设计进行观察与分析

不含无效状态



需要陷阱状态才能完全



如果缺失与 d 结点邻接的一条或两条弧，

这是其它情形（未清理，不予考虑）

d 为一个陷阱状态

DFA的一般形式

按照DFA $A=(Q,\Sigma,\upsilon,q_0,F)$ 的定义，$\mathrm{dom}(\upsilon)=Q\times\Sigma$，所以$A$ 是完全形DFA 。

将 A 中的无效状态合并为 1 个陷阱状态，得到DFA $A'=(Q',\Sigma,\upsilon',q_0,F')$，仍然是完全形，且：

A' 与 A 都定义同一语言；

$\#Q'\leqslant\#Q$；$\#F'\leqslant\#F$；

进一步删除A'陷阱状态，得到 DFA $A''=(Q'',\Sigma,\upsilon'',q_0,F')$

υ'' 的值为“无定义”表示自动机“突然死亡”并拒绝输入串。

那么，A'' 与 A' 都定义同一语言且 $\#Q''=\#Q'-1$。

一般常用 $q_\perp$ 作为陷阱状态，“无定义”用 $\perp$ 表示。

DFA的判定性质（算法表示）

- 算法：为几种形式的 DFA 都通用的判定性质

输入：DFA $A=(Q,\Sigma,\upsilon,q_0,F)$；输入串 $w\in\Sigma^*$。

输出：dfa(w) 为真表示接受 w，否则拒绝它。

```
int dfa(w) {
q = q0;                          //当前状态 q
x = w;                           //剩余串 x
while (x != ε) {
        x = ay;                                  //当前输入符号 a
        if((q = υ(q, a)) ∉Q)return 0;            //A 突然死亡
        x = y;                                   //修改剩余串
        }
return q∈F;                      //为真表示接受
}。
```

扩展转移函数

- 为了方便表示连续的转移，扩展转移函数 υ 为 $\tilde{\upsilon}$
- 归纳定义 $\tilde{\upsilon}$ 为

基础： $\tilde{\upsilon}(q, \varepsilon) = q$

归纳： $\tilde{\upsilon}(q, wa) = \upsilon(\tilde{\upsilon}(q, w), a)$

- $\tilde{\upsilon}(q\,, w)$ 表示DFA中存在唯一一条始端为 q、末端为 $\tilde{\upsilon}(q, w)$、标记为 w 的路径，即有 $\mathrm{PATH}[q, \tilde{\upsilon}(q, w), w]$
- 其中允许状态 $\perp$。

扩展转移函数举例

	0	1
$\rightarrow^*q_0$	q_0	q_1
$^*q_1$	q_2	q_1
q_2	q_2	q_2

$$\begin{aligned}
&\tilde{\upsilon}(q_0, 011)\\
&= \upsilon(\tilde{\upsilon}(q_0, 01), 1)\\
&= \upsilon(\upsilon(\tilde{\upsilon}(q_0, 0), 1), 1)\\
&= \upsilon(\upsilon(\upsilon(\tilde{\upsilon}(q_0, \varepsilon), 0), 1), 1)\\
&= \upsilon(\upsilon(\upsilon(q_0, 0), 1), 1)\\
&= \upsilon(\upsilon(q_0, 1), 1)\\
&= \upsilon(q_1, 1)\\
&= q_1
\end{aligned}$$

$$\begin{aligned}
&\tilde{\upsilon}(q_1, 011)\\
&= \upsilon(\tilde{\upsilon}(q_1, 01), 1)\\
&= \upsilon(\upsilon(\tilde{\upsilon}(q_1, 0), 1), 1)\\
&= \upsilon(\upsilon(q_2, 1), 1)\\
&= \upsilon(q_2\ 1)\\
&= q_2
\end{aligned}$$

	0	1
$\rightarrow^*q_0$	q_0	q_1
$^*q_1$	$\perp$	q_1

$$\begin{aligned}
&\tilde{\upsilon}(q_0, 101) =\\
&= \upsilon(\tilde{\upsilon}(q_0, 10), 1)\\
&= \upsilon(\upsilon(\tilde{\upsilon}(q_0, 1), 0), 1)\\
&= \upsilon(\upsilon(q_1, 0), 1)\\
&= \upsilon(\perp, 1)\\
&= \perp
\end{aligned}$$

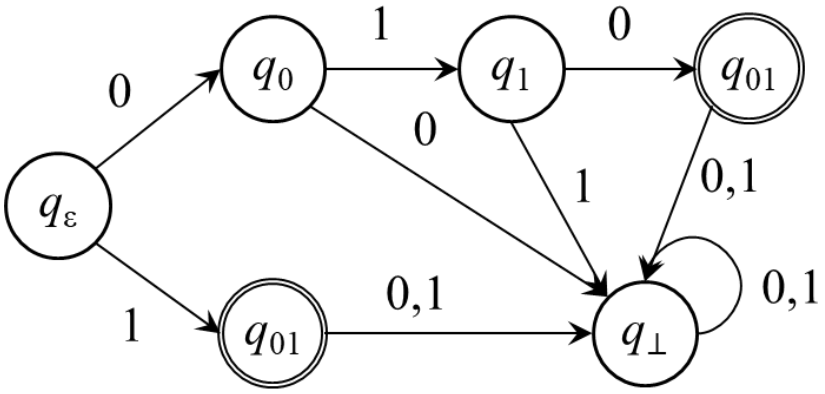
DFA A 中的无效状态 q

情形①：在 A 的状态图中，q 是从初始状态不可达的。

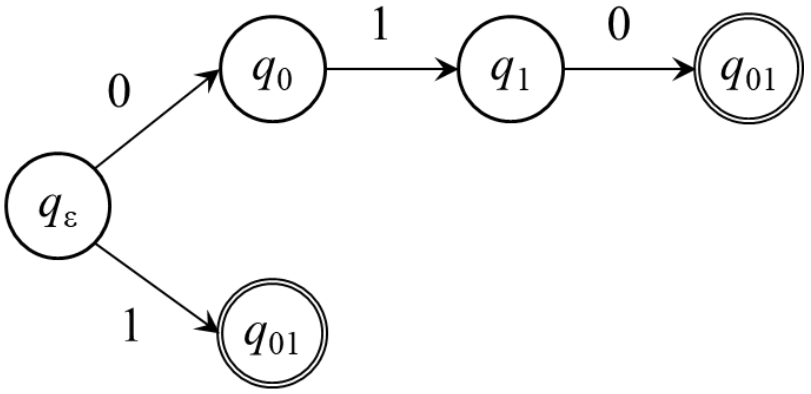
情形②：在 A 的状态图中，从 q 到任意接受状态都不可达。

情形①和②都表明 q 是无效状态，从而它不会出现在任意有效路径中。

任意有效路径上的状态都是有效状态。



(a)完全形



(b)最简形

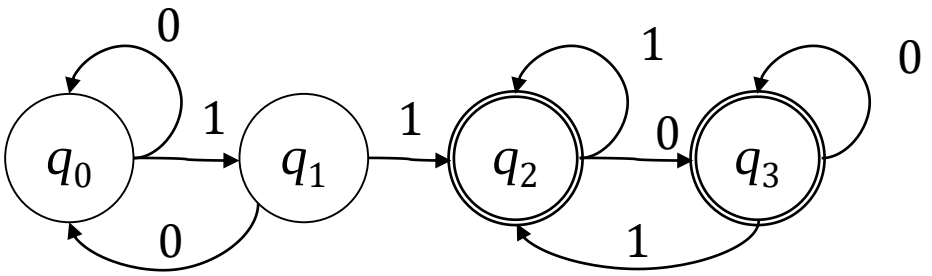
DFA定义的语言

- DFA $M = (Q, \Sigma, \upsilon, q_0, F)$
- $L(M) = \{w \in \Sigma^* \mid \tilde{\upsilon}(q_0, w) \in F\}$

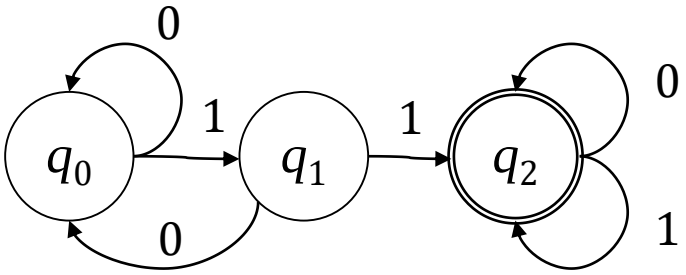
- DFA定义的语言是正则语言。
- 断言某语言是正则的，如果它是某个DFA的语言。

识别 {0,1} 上的含有子串 11 的串

	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_0	q_2
$^*q_2$	q_3	q_2
$^*q_3$	q_3	q_2

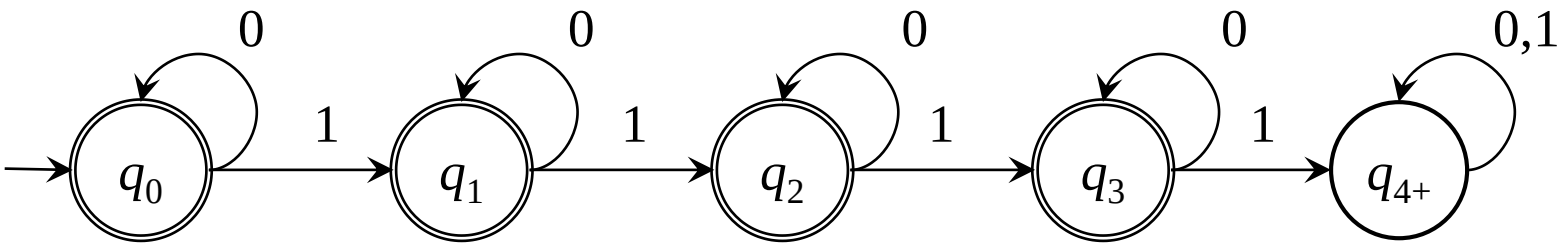


	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_0	q_2
$^*q_2$	q_2	q_2



{0,1} 上最多有三个 1 的串

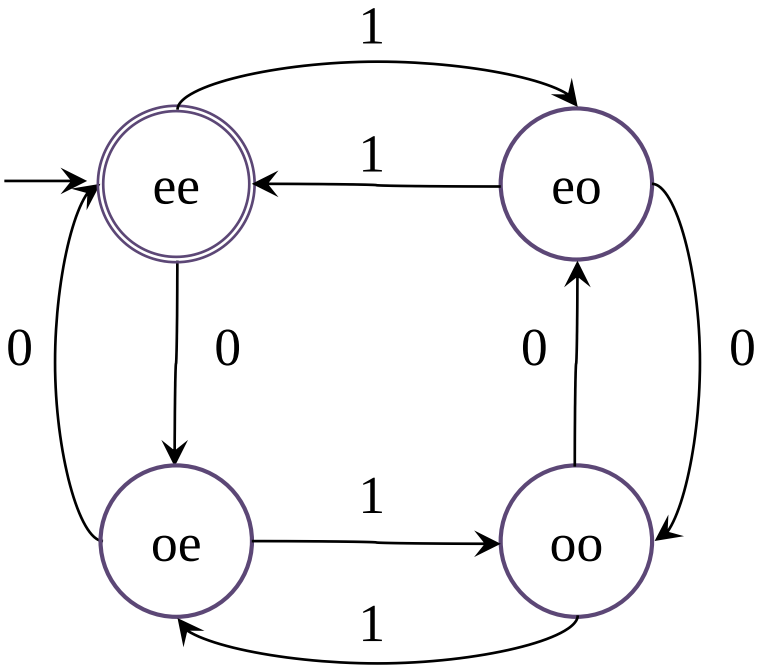
- 构造字母表$\{0, 1\}$上的DFA 接受所有最多含有三个1的串



例2.6

- {0,1}上的，含有偶数个0和偶数个1 的串的全集，是正则语言。

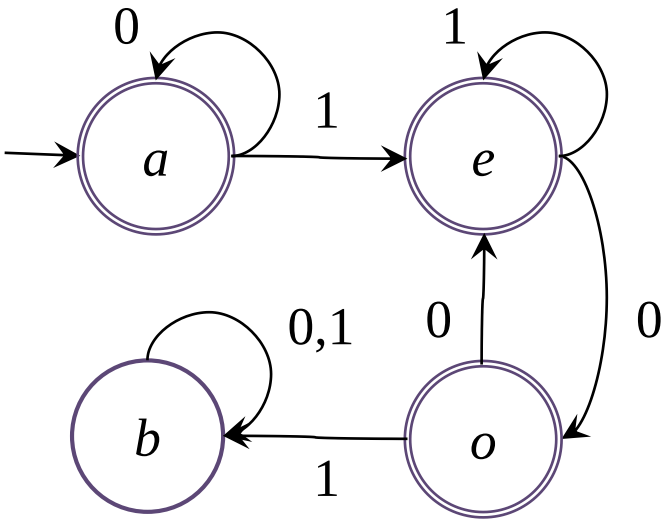
思路：输入串已消耗掉的部分只有四种情况，分别用状态来记住，继续读入符号将继续发生状态转移，当输入串消耗完以后，所达到的状态就决定了原输入串是什么情况，比如偶数个0和偶数个1



例2.7

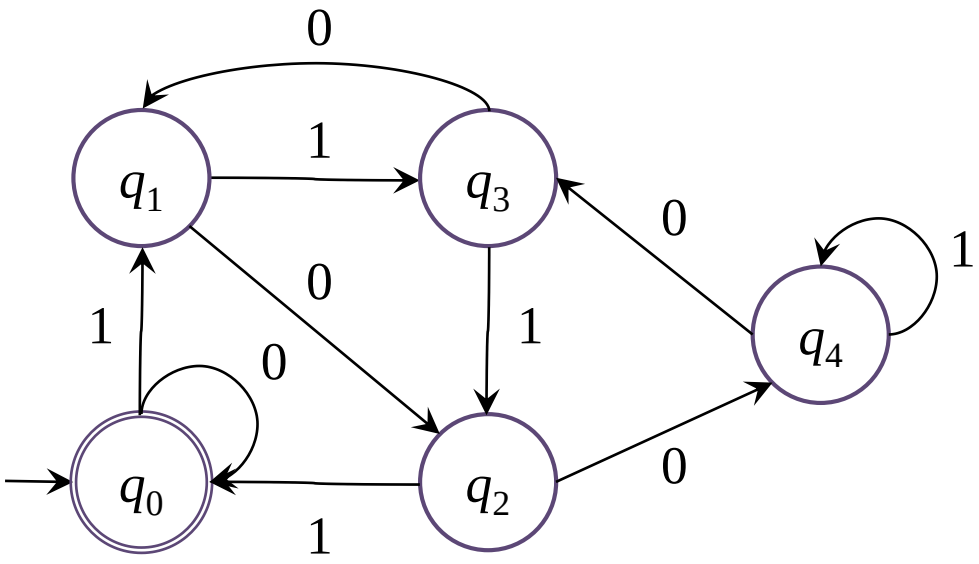
- {0,1}上不包含一对1且中间被奇数个0隔开的任何串。

  思路：排除包含模式10…01的串，该模式中的0为奇数个



例2.8

- 令$L=\{w\in\{0,1\}^*|w$看作二进制数的话能被5整除$\}$



分析：一个数除以5，其余数只能是0，…，4五种，因此我们以q_0，…，q_4分别表示这五种状态。因为接受能被5整除的数，故状态0既为初始状态，又为终结状态。接着，考虑二进制数在其串后增添0或1时，状态的转化情况。在二进制串后添1位，即可理解为将先前的串值乘以二再加上所添的数值。那么，串尾添数后新的数值模5的余数便可以计算出来。即可以得到添0或1后的新的状态。

归纳{0,1}上语言的DFA设计题目

- 前缀有这个、没这个、同时有这两个、或者有这个或者那个；
- 子串同上；
- 后缀同上；
- 汇总性：有 n 个 1、没有 n 个 1、有 n 个 1 和 m 个 0、没有 n 个 1 同时也没有 m 个 0；
- 计算性：被 5 整除、是素数、是奇数、是偶数

- 整数、定点数

- 对题目要求：典型且简单

用状态作记忆（例2.9）

- $\{0,1\}$ 上以 01 结尾的串
- 用四个状态记住输入串的前缀的最后两位（已消耗串的最后两位）。

- q_{00}
- q_{01}
- q_{10}
- q_{11}

	0	1
$\rightarrow q_\varepsilon$	q_0	q_1
q_0	q_{00}	q_{01}
q_1	q_{10}	q_{11}
q_{00}	q_{00}	q_{01}
$^*q_{01}$	q_{10}	q_{11}
q_{10}	q_{00}	q_{01}
q_{11}	q_{10}	q_{11}

用状态作记忆的局限性（例2.10）

- $\{0, 1\}$上以101结尾的串

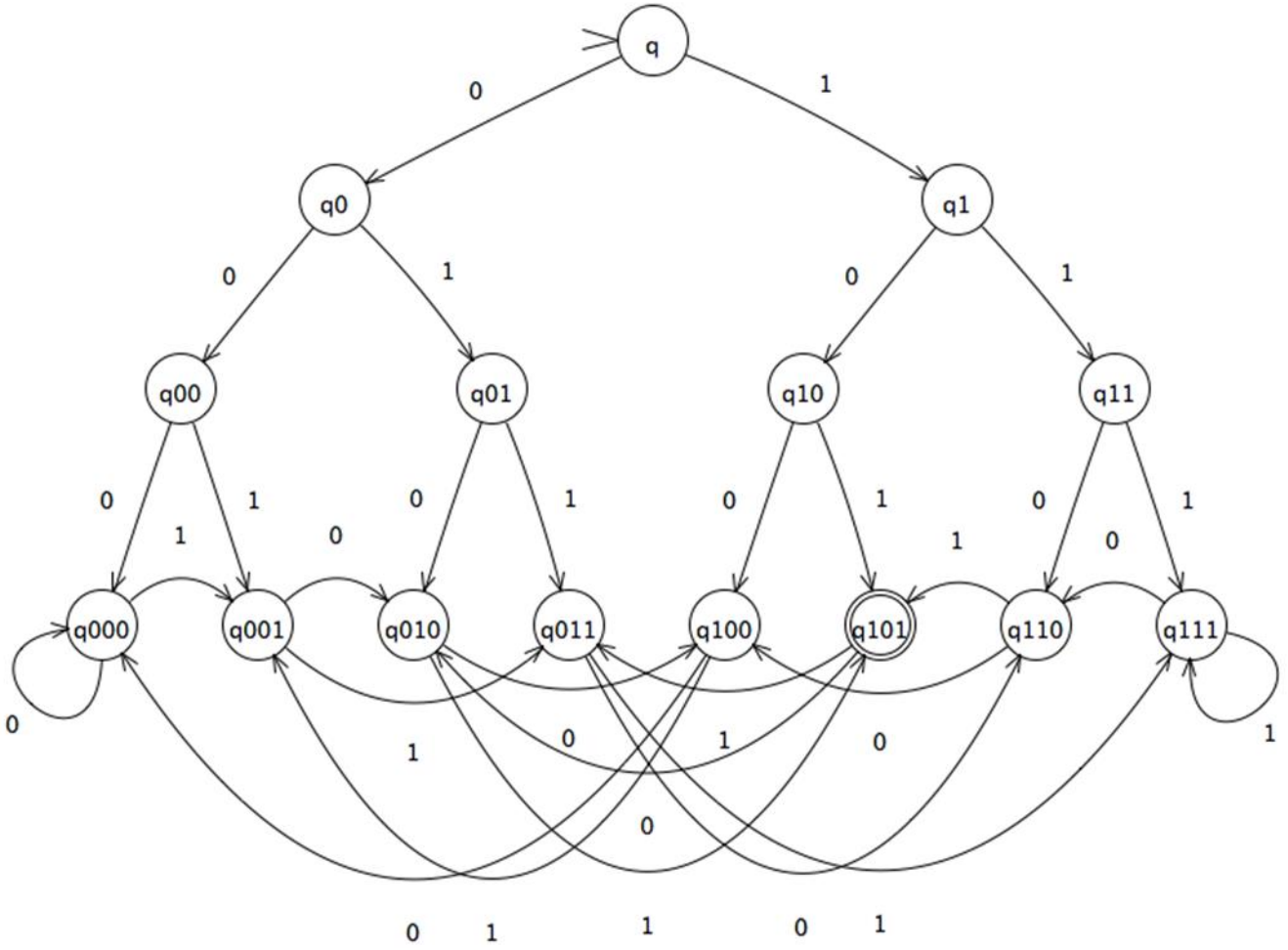
用八个状态记住已消耗串的最后三位。

q_{000}

q_{001}

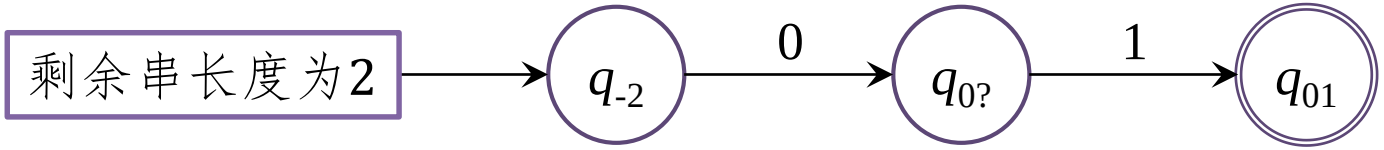
…

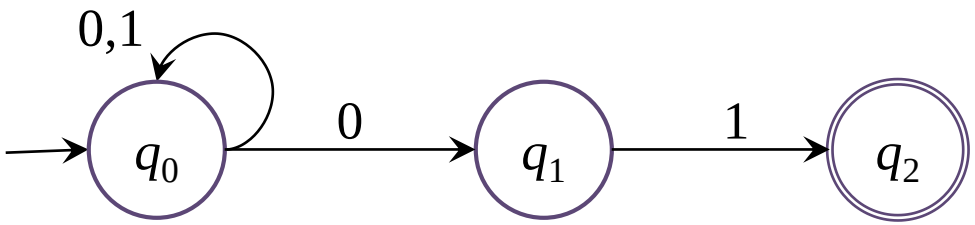
q_{111}



引入不确定性来避免状态爆炸

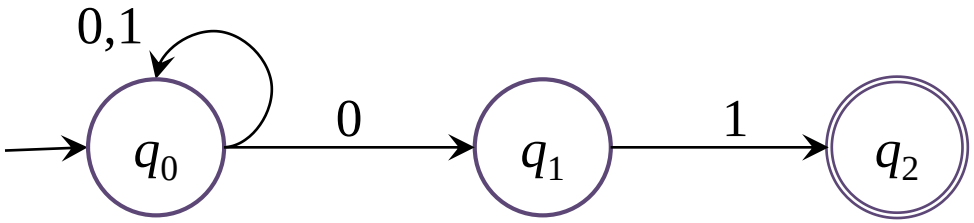
- 猜测

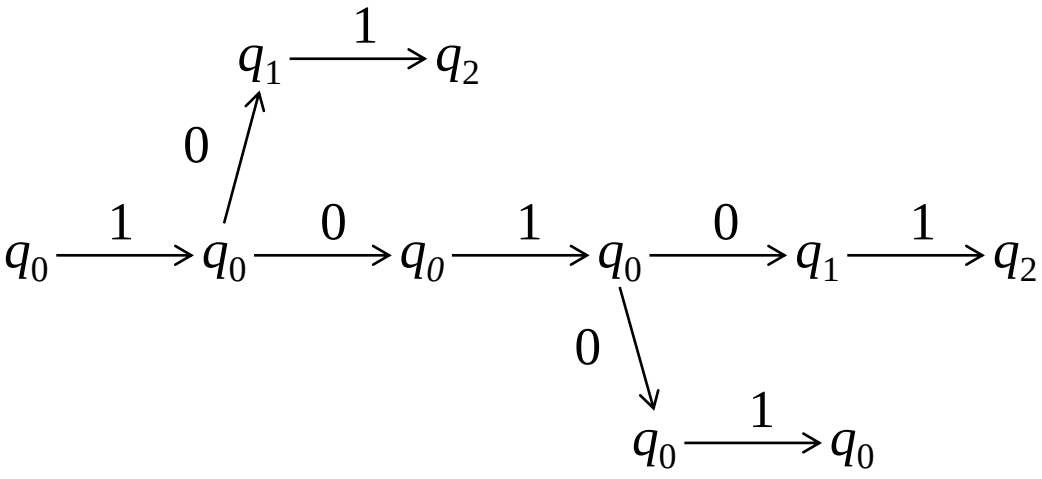




穷举与猜测

- 输入串10101





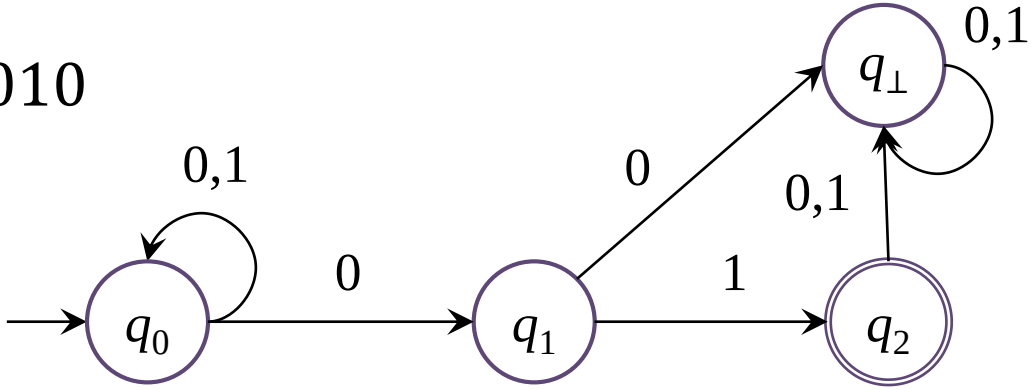
死亡

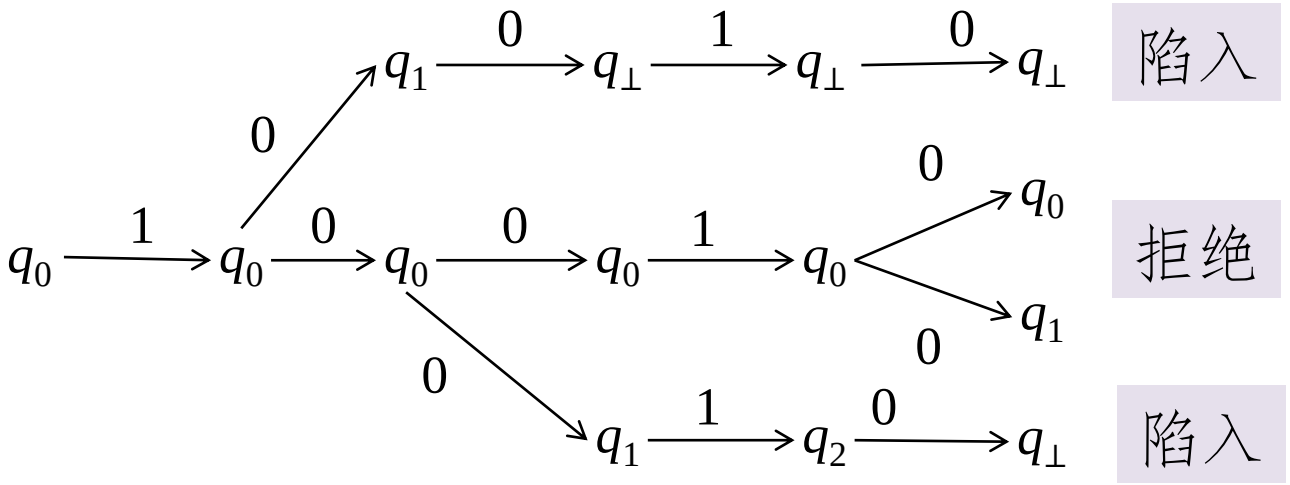
接受

拒绝

穷举与猜测

- 输入串10010





线索穷举

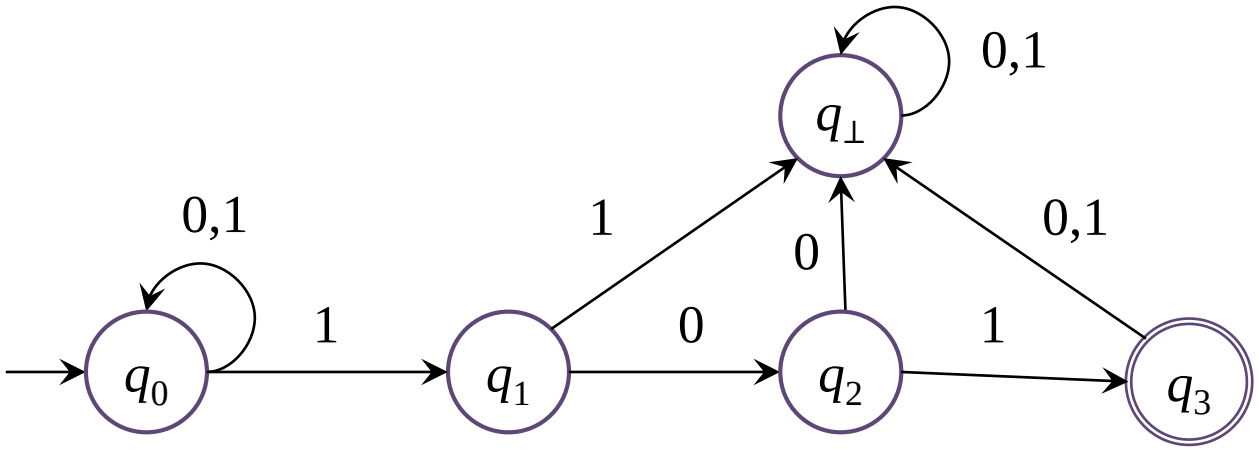
- 对于输入串，线索是自动机的一条以初始状态为起点的状态转移路径，该路径正好消耗掉输入串的某个前缀。线索穷举是有穷自动机的运行方式
  - DFA运行时只有一条线索（路径）。
  - NFA运行时有有穷个线索。
- 对于给定的输入串 w，只要有一个线索成功，则 w 被自动机接受，否则不被接受
  - 成功的线索：始端为初始状态，标记为 w，末端为接受状态。
- 在前例中，
  - 以 101 结尾的简化型NFA对于输入串 110101 有 5 个线索，其中有一个成功。
  - 以 101 结尾的完全型NFA对于输入串 110010 有4 个线索，均不成功。
  - 以 101 结尾的完全型NFA对于输入串 110101 有几个个线索？成功者为何？

NFA的线索穷举成为一个树

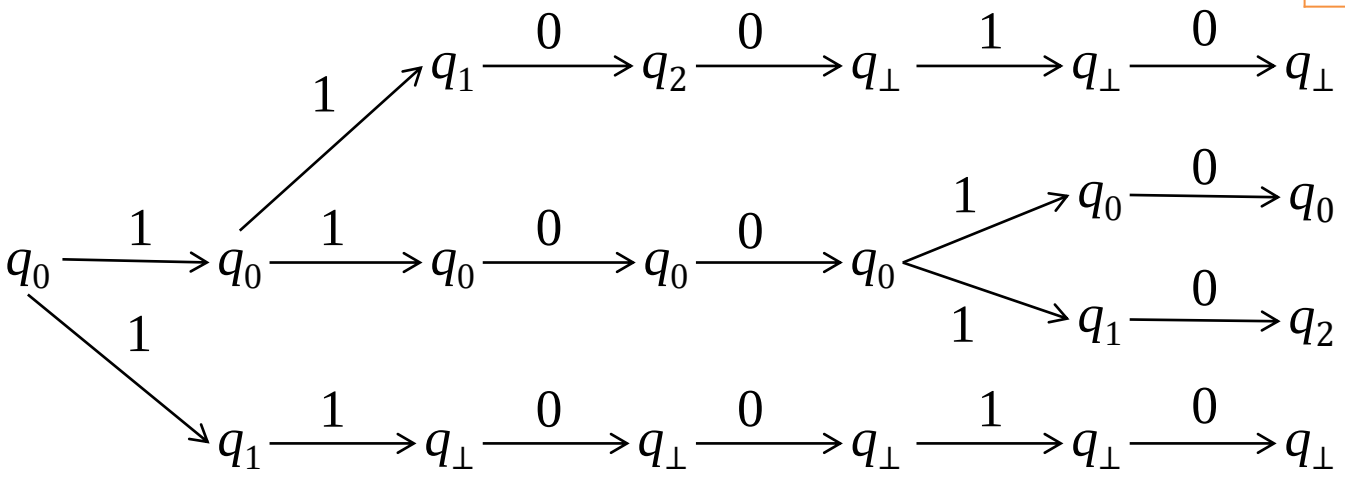
- w为输入串，w 的第 i（$i \geqslant 0$）个前缀就是长度为 i 的前缀，记为 i-前缀，等于 $\pi(w, i)$
- w=hello-world
- w 的前缀：ε，h，he，hel，hell，hello，hello-，hello-w，...，hello-world
- w 有 $|w|+1$ 个前缀
- 给定输入串的线索穷举树►
- 根为初始状态，层数为 0
- 从根到每一个叶子的路径都是一条线索（每个线索都在树中恰有一条从根到叶子的路径对应）
- 第 i 层（$i>0$）的状态集合=T_i
- 以根为起点，无论沿着那条线索，都消耗掉输入串的 i-前缀并转移到 T_i中的状态
- T_i 就是活动状态集，也是 w 的 i-前缀的当前状态集。

线索穷举树举例

输入串: 110010

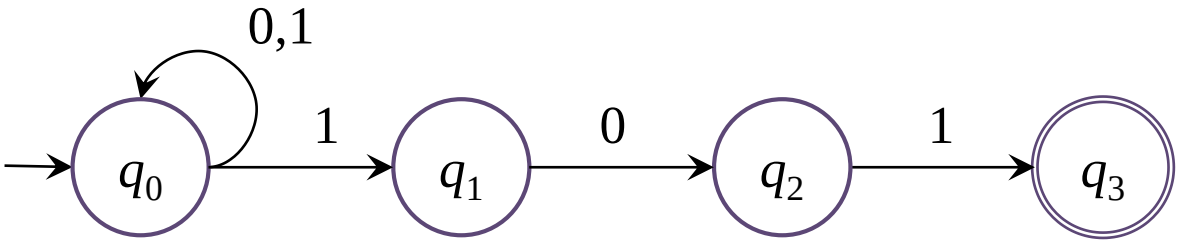


i	T_i	i前缀
0	$\{q_0\}$	ε
1	$\{q_0,q_1\}$	1
2	$\{q_0,q_1,q_\perp\}$	11
3	$\{q_0,q_2,q_\perp\}$	110
4	$\{q_0,q_\perp\}$	1100
5	$\{q_0,q_1,q_\perp\}$	11001
6	$\{q_0,q_2,q_\perp\}$	110010

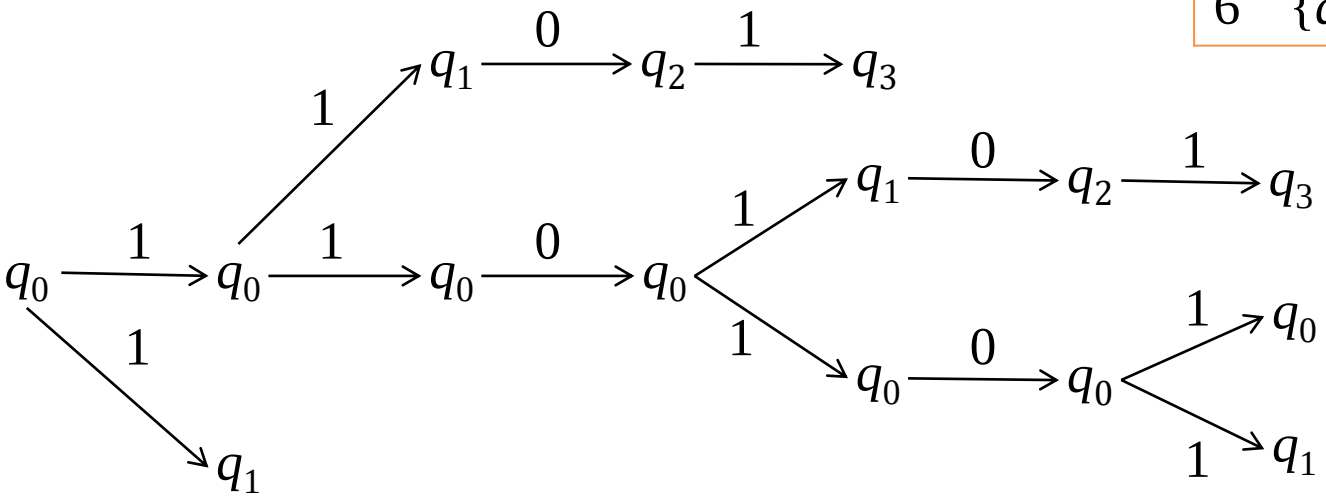


线索穷举树举例

输入串: 110101



i	T_i	i前缀
0	$\{q_0\}$	ε
1	$\{q_0,q_1\}$	1
2	$\{q_0,q_1\}$	11
3	$\{q_0,q_2\}$	110
4	$\{q_0,q_1,q_3\}$	1101
5	$\{q_0,q_2\}$	11010
6	$\{q_0,q_1,q_3\}$	110101

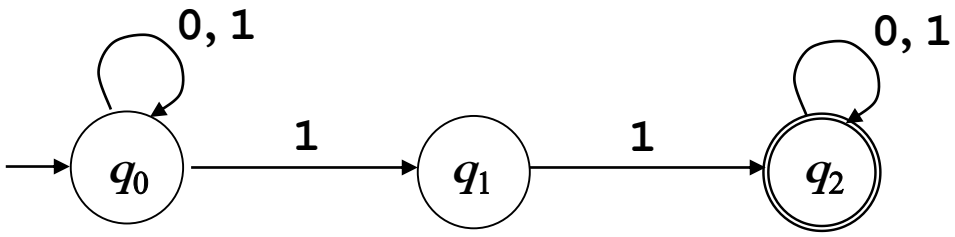


NFA例

- 构造字母表 $\{0, 1\}$ 上的NFA 接受含有子串 11 的串。

10011010, 11001, 11101　　接受

ε, 000, 010101　　拒绝



NFA与DFA有什么不同？

- DFA：从同一个状态射出的同标记箭弧最多有一条。
- NFA：从同一个状态射出的同标记箭弧可以有多条。

- ε-NFA：有 ε 转移的NFA。
- ε 转移：状态间的 ε 转移不消耗输入符号。

- 在2.3节介绍NFA、
- 在2.4节介绍ε-NFA
- 之后，二者不再区分，统称NFA

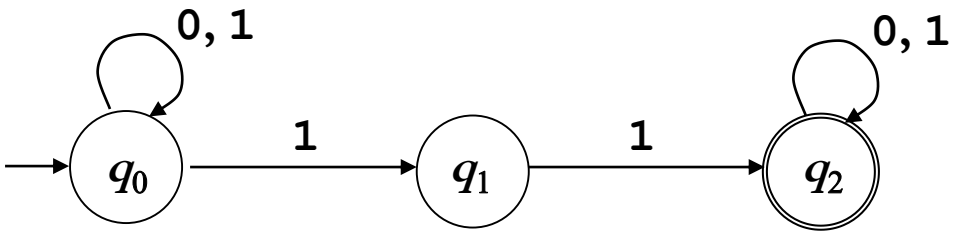
表示NFA：代数、转移图、转移表

- NFA $A=(Q, \Sigma, \upsilon, q_0, F)$
  - 状态集 $Q=\{q_0, q_1, q_2\}$
  - 字母表 $\Sigma=\{0, 1\}$
  - 转移函数υ: $Q\times\Sigma\rightarrow 2^Q$
  - 初始状态$q_0\in Q$
  - 接受状态集合 $F=\{q_2\}\subseteq Q$

$$\upsilon=\{((q_0,0),\{q_0\}),((q_0,1),\{q_0,q_1\}),((q_1,0),\{\}),((q_1,1),\{q_2\}),((q_2,1),\{q_2\}),((q_2,1),\{q_2\})\}$$

NFA $A=(\{q_0,q_1,q_2\},\{0,1\},\upsilon,q_0,\{q_2\})$

两种定义：
转移函数υ是映射； υ允许$\perp$值
用$\upsilon(q,a)=\perp$表示$\upsilon(q,a)$无定义。



	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\{q_2\}$	$\{q_2\}$

NFA的扩展转移函数

- 为了方便表示读入一个串所发生的连续转移，扩展转移函数 υ 为 $\tilde{\upsilon}$，$\tilde{\upsilon}$ 被称为扩展转移函数。

q 是NFA的状态，
对于串$w=a_1a_2...a_n$，如果，
$T_0=\{q\}$，
$T_i=\cup p\in T_{i-1}\cdot \upsilon(p, a_i)$, $1\le i\le n$，
那么，$\tilde{\upsilon}(q, w)=T_n$

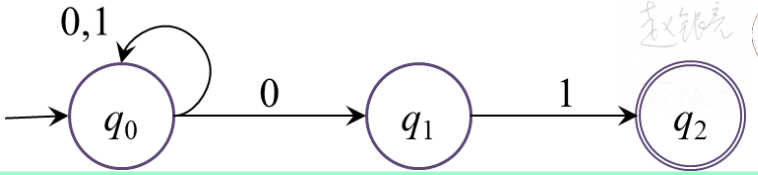
- 用归纳方式定义 $\tilde{\upsilon}$ 为

基础： $\tilde{\upsilon}(q, \varepsilon) = \{q\}$

归纳： $\tilde{\upsilon}(q, wa) = \cup p\in\tilde{\upsilon}(q, w)\cdot\upsilon(p, a)$

- $\tilde{\upsilon}(q, w)$：对于始端为 q，标记为 w 的所有路径，由它们的末端组成的集合。

例2.11的示例

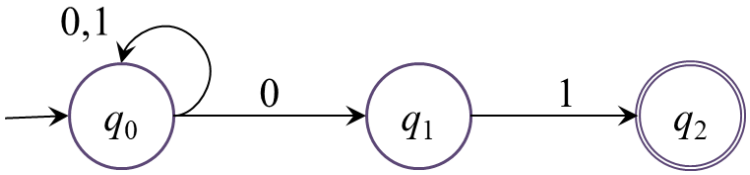


$$\tilde{\upsilon}(q_0, 01011) = \cup p \in \tilde{\upsilon}(q_0, 0101) \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in \tilde{\upsilon}(q_0, 010) \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in [\cup p \in \tilde{\upsilon}(q_0, 01) \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in [\cup p \in [\cup p \in \tilde{\upsilon}(q_0, 0) \cdot \upsilon(p, 1)] \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in [\cup p \in [\cup p \in [\cup p \in \tilde{\upsilon}(q_0, \varepsilon) \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in [\cup p \in [\cup p \in [\cup p \in \{q_0\} \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in [\cup p \in [\cup p \in \{q_0, q_1\} \cdot \upsilon(p, 1)] \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in [\cup p \in \{q_0, q_2\} \cdot \upsilon(p, 0)] \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in [\cup p \in \{q_0, q_1\} \cdot \upsilon(p, 1)] \cdot \upsilon(p, 1)$$
$$= \cup p \in \{q_0, q_2\} \cdot \upsilon(p, 1)$$
$$= \{q_0\}$$

$$\tilde{\upsilon}(q_0, \varepsilon) = \{q_0\}$$
$$\tilde{\upsilon}(q_0, 0) = \cup p \in \tilde{\upsilon}(q_0, \varepsilon) \cdot \upsilon(p, 0) = \{q_0, q_1\}$$
$$\tilde{\upsilon}(q_0, 01) = \cup p \in \tilde{\upsilon}(q_0, 0) \cdot \upsilon(p, 1) = \upsilon(q_0, 1) \cup \upsilon(q_1, 1) = \{q_0, q_2\}$$
$$\tilde{\upsilon}(q_0, 010) = \cup p \in \tilde{\upsilon}(q_0, 01) \cdot \upsilon(p, 0) = \upsilon(q_0, 0) \cup \upsilon(q_2, 0) = \{q_0, q_1\}$$
$$\tilde{\upsilon}(q_0, 0101) = \cup p \in \tilde{\upsilon}(q_0, 010) \cdot \upsilon(p, 1) = \upsilon(q_0, 1) \cup \upsilon(q_1, 1) = \{q_0, q_2\}$$
$$\tilde{\upsilon}(q_0, 01011) = \cup p \in \tilde{\upsilon}(q_0, 0101) \cdot \upsilon(p, 1) = \upsilon(q_0, 1) \cup \upsilon(q_2, 1) = \{q_0\}$$

NFA的语言

- NFA $A = (Q, \Sigma, \upsilon, q_0, F)$ 的语言，
- $L(A) = \{w \in \Sigma^* \mid \tilde{\upsilon}(q_0, w) \cap F \neq \emptyset\}$。
- 这个定义也显示，只要有一个成功线索，就表示接受该输入串。
- 证明例2.11接受 {0,1} 上 2-后缀等于 01 的串集合（语言）
  - 命题一：$w \in \Sigma^*$, $q_0 \in \tilde{\upsilon}(q_0, w)$；
  - 命题二：$q_1 \in \tilde{\upsilon}(q_0, w)$，当且仅当 w 以 0 结尾；
  - 命题三：$q_2 \in \tilde{\upsilon}(q_0, w)$，当且仅当 w 以 01 结尾。



小结

- DFA：定义及表示、判定性算法、扩展转移函数、语言与DFA构建。
- NFA：定义及表示，判定性算法、扩展转移函数、线索穷举，活动状态集

- 作业：
- 2.5节习题 2.1(2); 2.3–2.4